- *Scaffolding of columns*—EF Core uses conservative values for things like `string` columns by allowing strings of large or unlimited length. In practice, you may want to restrict these and other data types to sensible values.
- *Validation*—You can decorate your entities with `DataAnnotations` validation attributes, but EF Core won't validate the values automatically before saving to the database. This behavior differs from EF 6.x behavior, in which validation was automatic.
- *Handling concurrency*—EF Core provides a few ways to handle *concurrency,* which occurs when multiple users attempt to update an entity at the same time. One partial solution is to use `Timestamp` columns on your entities.
- *Handling errors*—Databases and networks are inherently flaky, so you'll always have to account for transient errors. EF Core includes various features to maintain connection resiliency by retrying on network failures.
- *Synchronous vs. asynchronous*—EF Core provides both synchronous and asynchronous commands for interacting with the database. Often, async is better for web apps, but this argument has nuances that make it impossible to recommend one approach over the other in all situations.

EF Core is a great tool for being productive in writing data-access code, but some aspects of working with a database are unavoidably awkward. Database management is one of the thorniest problems to tackle. Most web applications use some sort of database, so the following problems are likely to affect ASP.NET Core developers at some point: